

Diskless Viewstamped Replication with Unbounded Crash-Stop Reincarnation

Simon Massey

Abstract—Small replicated metadata should not require a disk flush on the critical path. Unbounded Viewstamped Replication Revisited is a protocol design for embedding strong consistency with volatile normal-path replication and crash-stop reincarnation. A cleanly stopped process restarts with its identity and flushed state. A crashed process permanently retires its old identity and rejoins under a fresh identity, obtaining state and voting authority through reconfiguration. Its organising mechanism is leader overlap: where the leader is the sole intersection of the active decision quorum and the next prepare quorum, it withholds its own promise so that preparation can proceed while the active quorum remains usable. Five-voter replacement uses six committed batches, including exact doubling and halving. A constructive Rust solver also computes paths between arbitrary weighted-majority configurations with an available majority at each endpoint. The Lean development proves weighted reachability and a phantom-identity casting-vote construction, alongside agreement and identity exclusion for a three-node schedule and its five-voter extension. The addendum published with the code separates the proof contracts and executable checks.

Index Terms—Crash-stop, leader overlap, reconfiguration, strong consistency, Viewstamped Replication.

I. INTRODUCTION

Oki and Liskov introduced Viewstamped Replication in 1988 [1]. Liskov and Cowling’s 2012 Viewstamped Replication Revisited (VRR) eliminated disk writes during normal processing and view changes [2]. Replication therefore retained the state needed for agreement without adding a local disk flush to each update. The remaining difficulty was how to bring a replica back after loss of that state.

Michael *et al.* exposed a safety failure in VRR’s recovery mechanism in 2016 [3, Sec. 4.2 and Appendix A.1]. A replica can send a view-change commitment, crash, and return in an older view having forgotten the commitment. It can then help commit an operation that a later leader loses when delayed view-change messages arrive. Their counterexample breaks linearizability, including when channels preserve message order. The authors provide a virtual stable storage construction to support diskless crash recovery. Their result makes the problem precise: restoring a replica’s data must also preserve the commitments made by its identity.

Unbounded Viewstamped Replication Revisited (uVRR) repairs this failure by making a crash final for the protocol identity. The crashed identity never resumes execution. The process that replaces it joins under a fresh identity, receives state from the cluster, and acquires voting authority through committed membership changes. Messages sent before the crash remain messages of the old identity; the new process

cannot add fresh votes to that identity’s history. A cleanly stopped process can instead restart under its existing identity because it has preserved its protocol state. Startup fencing and clean shutdown use durable writes, keeping those writes outside normal replication. Loss of volatile state therefore leads to replacement of a member: crash recovery becomes crash-stop reincarnation.

Turner supplies the construction that connects this lifecycle to continued replication [4]. Where the leader is the sole intersection of an active decision quorum and the next prepare quorum, it can withhold its own promise while collecting the others. The active quorum remains usable during that preparation; the leader completes the handover with its own local promise. We use this leader overlap to organise reincarnation. For three unit-weight voters, replacement takes two committed batches. For five, the weighted schedule has six, including exact doubling and halving. The addendum published with the code [15] gives the proof ladder, a general configuration-path constructor and an abstract casting-vote completion using fresh identities. Turner’s agreement argument supplies the basis for composing safe adjacent configuration boundaries.

Together, these ideas retain VRR’s diskless normal path while giving failed members a route back through fresh identities. The intended setting is agreement over small amounts of metadata embedded in a distributed application [5]. Such a service can coordinate membership or advisory-lock metadata without putting synchronous persistence on every update’s critical path. Its normal replication need not wait for a disk flush or batch updates to amortise one.

Section II develops the lifecycle and leader-overlap construction. Section III starts with three-voter Crash-Stop-Reincarnation and extends the argument to five voters, stating the history invariants needed for agreement, exclusion of the evicted voter and identity nonreuse. Section VI develops the metadata applications; the accompanying addendum gives the mathematical working and verification evidence. The name “unbounded” follows Turner’s treatment of pipelining through reconfiguration: agreement does not depend on a fixed bound on outstanding slots.

II. METHOD

A. Processes, state and identity

The agreement model is asynchronous and non-Byzantine. Messages may be delayed, and an old message retains the identity that issued it. Each incarnation has a logical identity distinct from its physical host. The lifecycle must ensure that an identity never produces new protocol actions after a replacement incarnation takes over.

A clean stop first quiesces protocol processing, then flushes the state required for restart, and finally records a durable stopped marker. On startup, a valid stopped marker permits the process to reload that state and enter *Restarting* with the same identity. Before sending protocol messages it durably records that it is running again. This boot fence prevents a later crash from being mistaken for another clean restart. A restarting process retains its promises and accepted state, so ordinary timeout processing is permitted by the design.

Without a valid clean-stop marker, startup enters *Joining* under a fresh identity. The old identity is crash-stopped. Cached state may reduce transfer work, but it provides no voting authority. The joiner asks the leader to evict the old incarnation and admit the new one; the leader can send missing state while coordinating the change. A joiner does not vote or initiate a view change. Its authority comes from the committed configuration, never from the fact that it shares a host with a former voter.

The implementation design uses replicated superblock markers and majority interpretation of valid marker copies. This paper abstracts that mechanism as correct clean-stop classification and durable identity fencing. The Lean result concerns the resulting lifecycle; it does not model the storage protocol itself.

B. Quorums across eras

An era determines the quorum families for a suffix of log slots. Write $\mathcal{Q}_e^I$ for its phase-one family and $\mathcal{Q}_e^{II}$ for its phase-two family. The history model requires the following within-era and adjacent-era intersections:

$$q \cap r \neq \emptyset \quad (q \in \mathcal{Q}_e^{II}, r \in \mathcal{Q}_e^I), \quad (1)$$

$$q \cap r \neq \emptyset \quad (q \in \mathcal{Q}_e^I, r \in \mathcal{Q}_{e+1}^{II}). \quad (2)$$

The direction in (2) matters: the earlier phase-one quorum must meet the later decision quorum. These conditions are part of Turner's era-based agreement argument [4].

Our concrete schedule uses the same strict weighted-majority family for both phases. If $w_e(n)$ is a nonnegative integer weight on a finite support, a quorum q satisfies

$$2 \sum_{n \in q} w_e(n) > \sum_n w_e(n). \quad (3)$$

The weighted proof establishes the requisite intersections when successive configurations differ by at most one unit of total absolute weight. Membership records for zero-weight nodes may change in the same batch without altering (3). A crash itself does not change any weight; only committed configuration commands do.

III. THREE-NODE CRASH-STOP-REINCARNATION

Suppose a leader ℓ uses ballot b with an active decision quorum q . To prepare ballot $b' > b$ for the next era, it chooses a prospective phase-one quorum q' with

$$q \in \mathcal{Q}_e^{II}, \quad q' \in \mathcal{Q}_{e+1}^I, \quad q \cap q' = \{\ell\}. \quad (4)$$

This is Turner's casting-vote condition [4]. The leader first sends prepare requests only to $q' \setminus \{\ell\}$. Their promises do

not disable an acceptor in q . The leader withholds its own promise and may continue proposing at b , provided no other event invalidates the old ballot's guards.

After receiving those promises, the leader chooses the suffix boundary k and makes its own promise locally. Phase one is then complete without another network exchange for the leader's response. Proposals at b' must still obey the normal rule for preserving values reported by phase one. Figure 1 shows the message order.

Decision quorums follow the era of the *slot*, not simply the era associated with a ballot. Consequently, continuing to decide later-era slots at b also requires a quorum valid for those slots. In the concrete pivot below, the active quorum is a majority in both adjoining eras.

Table I is the three-node schedule: n_0 fails and is replaced by its fresh identity n_3 , while n_1, n_2 survive. A join inserts a zero-weight identity; an increment grants its voting weight. A leave removes the old identity after its weight reaches zero.

Each boundary preserves cross-era majority intersection by a unit change or exact scaling. Figure 1 illustrates the promotion boundary.

The leader stays live for client requests and drives a forced weight sequence from the voting-weights rules [12]: the exiting node's weight goes $1 \rightarrow 0$ (a Leave), the new identity joins at weight 0 (a Join), and the new identity goes $0 \rightarrow 1$. Each step is a unit-weight change, so the strict majorities of consecutive configurations overlap by the unit-change overlap argument and every intermediate era is quorum-safe on its own; a leader dying mid-sequence leaves only safe eras that a new leader can continue. Table I instantiates the sequence at unit weight for the three-voter cluster A, B, C with B the leader and C the crashed identity that returns as D : the start configuration is $(1, 1, 1, \cdot)$; era 1 drives the leader's batch $\text{DECREMENT}(C) + \text{JOIN}(D)$, reaching $(1, 1, 0, 0)$; era 2 drives $\text{INCREMENT}(D) + \text{LEAVE}(C)$, reaching $(1, 1, \cdot, 1)$. In the table a $-$ is not a weight: it marks an identity that is not a member of the configuration in that era, whereas 0 is the weight of a standby member that the cluster still holds in its order — which the reincarnated node is between the two eras while it is streamed preemptively. The old identity C is evicted by reincarnation, never a recovered amnesiac. If the sequence runs in the doubled scale of a weight-doubling plan, the return halve is integral only when all weights are even, so the corner sequence inserts one extra unit increment on the new identity before the halve. The leader streams the first reconfiguration, then client traffic, and preemptively streams to the reincarnated node as a weight-0 standby; messages to it are fine and messages from it are discarded by the standard membership check. If the leader crashes, the cluster must reach a stable leader before the reincarnated node forces its old identity's eviction.

Fig. 2 shows this forced sequence as a space-time chart drawn in the grammar of Turner's message diagrams [4]: four lifelines A, B (the leader), C , and D , time flowing down; a numbered diagonal vector per step; the bold cross where C crashes and is fenced, with its dead segment dashed below; the open circle where D 's lifeline begins; the dashed era-1 separator carrying $\text{DECREMENT}(C) + \text{JOIN}(D)$, with

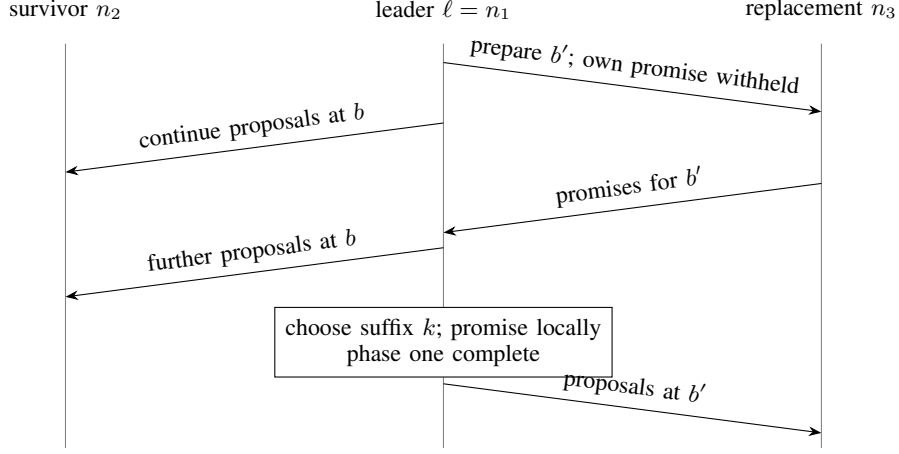


Fig. 1. Three-voter leader overlap at $E_1 \rightarrow E_2$: $q = \{n_1, n_2\}$ and $q' = \{n_1, n_3\}$. Each side outside the leader is a one-node minority. Original schematic of Turner's casting-vote construction [4]. Time runs downwards without a scale; acceptance replies, state transfer and unrelated traffic are omitted.

TABLE I
REINCARNATION FORCED WEIGHT SEQUENCE.

Era	A	B	C	D
start	1	1	1	–
era 1: DECREMENT (C) + JOIN (D)	1	1	0	0
era 2: INCREMENT (D) + LEAVE (C)	1	1	–	1

D joining at weight 0 as a standby and receiving the dashed preemptive stream; the era-2 separator carrying INCREMENT (D) + LEAVE (C), with D promoted to weight 1 and C leaving at weight 0; and the forming quorum $\{A, B\}$ promised in both eras, so no casting-vote pivot is exercised on this path. The chart is rendered from the tracked source diagrams/reincarnation.svg.

The addendum published with the code [15] provides a Lean theorem that composes an era-based agreement theorem with replacement weights and an abstract identity lifecycle. The work builds upon Turner's P2–P7 [4]: promises preserve their fences, reports identify prior acceptances, proposals preserve the selected history, and chosen values have quorum evidence. A well-founded total ballot order and the ballot/slot era conditions complete the contract. The addendum states each hypothesis and the exercise that connects operational executions to it.

Theorem 1 (Three-voter Crash-Stop-Reincarnation). *For a history using Table I, under the stated order, era and history conditions, any two values chosen for the same slot are equal. The first batch reduces the failed identity to zero voting weight and admits the fresh identity at zero; the second grants the fresh identity unit weight. The retired identity remains without voting authority when subsequent configurations preserve its exclusion. Along the stipulated identity lifecycle, a superseded identity never becomes current again. In a unit-weight three-voter configuration, any leader has a casting vote between two equal one-node minorities; at the promotion boundary, the surviving leader is the singleton overlap shown in Figure 1.*

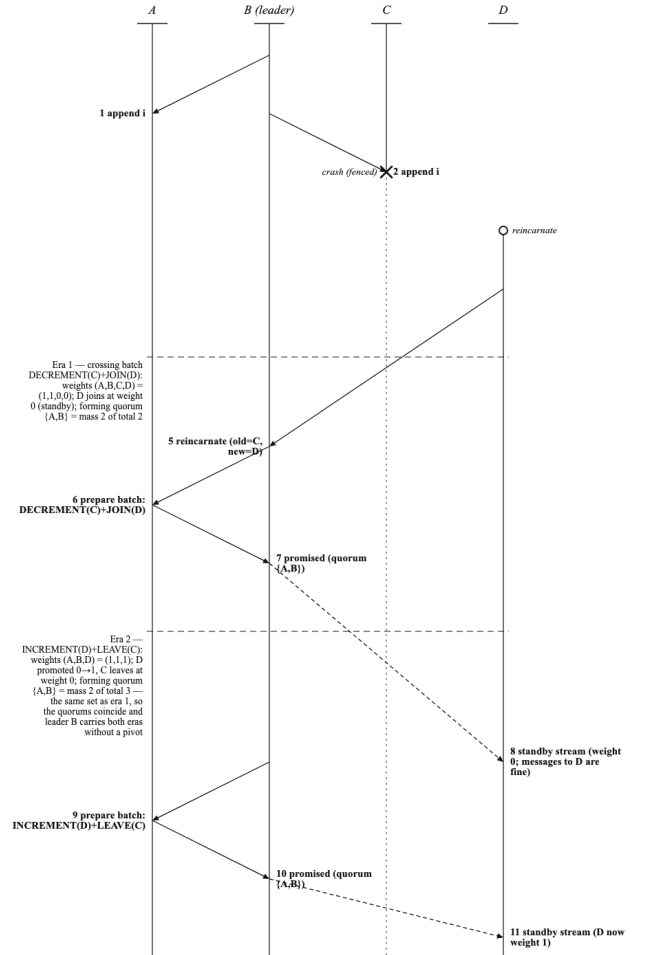


Fig. 2. The reincarnation forced sequence.

Proof. The totals in Table I are 3, 2, 3, so the majority thresholds are 2, 2, 2. Each boundary changes one weight by one; the integral weighted-overlap argument gives the within-era and adjacent-era intersections. Turner's agreement theorem

TABLE II
FIVE-NODE CRASH-STOP-REINCARNATION: SIX COMMITTED BATCHES.
ZERO DENOTES VOTING WEIGHT, INCLUDING IDENTITIES ABSENT FROM
MEMBERSHIP.

Stage	n_0	n_1	n_2	n_3	n_4	n_5
Initial	1	1	1	1	1	0
Double	2	2	2	2	2	0
Join, increment	2	2	2	2	2	1
Decrement old	1	2	2	2	2	1
Decrement, leave	0	2	2	2	2	1
Increment new	0	2	2	2	2	2
Halve	0	1	1	1	1	1

applies to those intersections and P2–P7. The two commands establish the stated weights, and identity fencing supplies nonreuse. At the promotion boundary, $\{n_1, n_2\}$ and $\{n_1, n_3\}$ meet only at leader n_1 ; their nonleader members form the two minorities. The leader can collect the replacement’s promise while withholding its own. The addendum gives the complete three-node working, including the abstract witness at the first boundary and the state-acquisition obligation. $\square$

IV. FIVE-NODE CRASH-STOP-REINCARNATION

For five original voters, n_0 is replaced by n_5 while $n_1, \dots, n_4$ survive. Table II gives the complete six-batch weighted schedule, including exact doubling and halving.

Both schedules are exercised by the Rust tests, and the addendum [15] gives the arithmetic for both schedules and the general construction of available paths and abstract leader overlaps.

With all five unit voters responding, a leader has the casting pair $\{\ell, a, b\}$ and $\{\ell, c, d\}$. After one voter fails, obtaining the corresponding responding split requires more care. The mechanical transformations in Table II organise the weighted replacement while preserving the agreement invariant; the addendum supplies their safety argument and the distinction between abstract witnesses and protocol receipts.

Theorem 2 (Five-voter Crash-Stop-Reincarnation). *For a history using Table II, under the same order, era and P2–P7 conditions, any two values chosen for the same slot are equal. After the fourth batch the old identity has zero weight; after the sixth the replacement and all four survivors have unit weight. Preserving exclusion in subsequent configurations and enforcing the identity lifecycle retain the old identity’s retirement.*

Proof. Doubling and exact halving preserve the majority family. Each of the four intervening boundaries changes one unit of voting mass. The same local intersection argument therefore applies six times. Induction over these boundaries establishes the era intersection invariant, and Turner’s theorem gives agreement. Evaluation of the commands gives the stated final weights. $\square$

V. EVIDENCE

The addendum [15] maps these arguments to Lean 4 [16], bounded exhaustive enumeration, Rust property tests and

protocol tests. It includes the existing specialised machine theorem’s precise scope, the general lemmas used above, and the reproduction commands. Configuration reachability, abstract casting witnesses and the operational history contract are stated separately so each result can be checked at its own layer.

VI. DISCUSSION

A. Applications

The intended deployment is an embedded authority for small metadata, with ordinary replication performed in memory.

a) Short-lived advisory locks and leader leases: A replicated state machine can order acquisition, renewal and expiry decisions over a small record containing an owner, generation and expiry policy. An expiry command for an old generation must not revoke a newer grant. If a client uses ownership to control an external resource, that resource may also need to reject stale generations. A real-time leader lease additionally needs explicit clock and timing assumptions.

b) A membership authority for another algorithm: An application whose bulk algorithm uses gossip or another weaker consistency model may still need one agreed version of cluster membership, ownership assignments or configuration changes. An embedded replicated state machine can supply that authority, while the bulk algorithm disseminates or consumes its decisions.

c) The configuration as replicated state: In the Paxos family the configuration itself is replicated consensus state: Lamport, Malkhi and Zhou define a reconfigurable state machine whose configuration is a state component changed only by an executed reconfiguration command [13], and Vertical Paxos instead commits each new configuration to an auxiliary configuration master, which determines the set of acceptors of every configuration [14]. This design keeps that commitment inside the application: reconfiguration batches commit in the same slot stream as client values.

d) The CORFU precedent: CORFU provides a concrete precedent for compact configuration metadata alongside a larger storage system [6]. Its reconfiguration is patterned after Vertical Paxos; Section 2.5 describes storage units acting as acceptors in a Paxos-like protocol. Section 2.6 estimates that adding 0.1% writes keeps projection metadata below 25 MB for a 1 TB cluster under an adversarial workload. The described implementation uses a shared network drive for agreed projections.

e) External coordination services: ZooKeeper and etcd are established ways to supply coordination separately from an application. ZooKeeper offers linearizable state changes and per-client ordering, while local reads have weaker semantics [7]. etcd documents strict serializability for its key-value operations and distinguishes default linearizable reads from optional serializable reads [8]. An application must choose the operation semantics it actually needs.

A separately operated service also brings deployment and resource costs. For scale, etcd’s version 3.6 hardware guide gives a small-cluster example with dedicated two-vCPU, 8

GB AWS machines [9]. etcd can itself be embedded in a Go application [10]. The proposed combination here is small application-owned state, volatile normal-path replication, and replacement through fresh identities and overlapping quorums.

B. Scope

a) Implementation correspondence: The theorems assume protocol-history invariants. Their composition with the era-changing operational model is left as an exercise to the reader. The addendum [15] makes that exercise verifiable by stating its mapping, base case and preservation obligations, including clean-stop classification, identity fencing and state transfer before voting.

b) Scope and availability: Tables I and II replace one voter among three and five respectively. The addendum constructs paths between arbitrary weighted-majority endpoints with available majorities. Agreement follows from the history contract; progress additionally uses available recipients and a stable leader. Fresh identities permit repeated replacement while each committed boundary preserves the invariant.

c) Meaning of strong consistency: The mechanised result is per-slot agreement. A complete service-level linearizability argument also needs ordered execution, appropriate read semantics and a mapping between client operations and log entries. Lease guarantees require the additional conditions stated above.

VII. THE DEMONSTRATION: LUNET-LOCKS

The demonstration is `lunet-locks`, a runnable, fixed-membership advisory-lock service. It orders lock requests with `vrr-core`, the sans-IO Viewstamped Replication Revisited core that accompanies this formalisation: one total function maps a tick, an input message, and a state to a new state and the messages to send, and the host owns time, transport, storage, packetisation, threading, and naming. Clients speak newline-delimited JSON over TCP; replicas exchange the core’s binary datagrams over raw UDP; the service process is LuaJIT and libuv (the Lunet runtime, version-pinned and hash-verified), and a small Rust adapter drives the core and owns the lock state machine.

A. Components

The adapter instantiates the concrete core replica over a segmented log with weighted-majority quorums, running volatile stability: protocol state lives in quorum memory, an empty journal is provisioned at boot, and the only bytes the service ever fsyncs locally are the recovery nonce file. The adapter retains the tick clock (a monotonic value clamped against wall-clock regression), exactly-once reply correlation keyed by the client’s message identifier, and guard checks that re-validate every replicated operation entry — in Prepare, DoViewChange, StartView, RecoveryResponse, and NewState messages — against the lock service before the entry reaches the core. The Teal/Lunet wrapper owns sockets, TCP framing, peer source validation, and forwarding; a UDP peer envelope carrying a membership fingerprint guards the fixed membership at the transport boundary, so differently configured

clusters cannot merge. An append-only lock-event journal, a REST/WebSocket feed, and a console are observability only: the journal never participates in replication or recovery, and a journal error disables journaling without affecting the service path.

B. The rejoin path implemented

Every boot enters fenced recovery stamped with a fresh durable nonce. The nonce file is updated atomically — write, fsync, rename, parent-directory sync — and the new value is used as the recovery input’s tick, so per-node monotonicity holds across the two clock sources. Recovery needs a live quorum: a restarted node rejoins from quorum memory, installing the latest fenced view’s history and re-applying its committed suffix. A rolling single-node restart therefore recovers from the surviving members, while a simultaneous full-cluster loss forfeits whatever the quorum held, and an operator re-bootstrapping a lost cluster must do so only after all outstanding leases can no longer be valid.

Membership is fixed for one process lifetime, so the reincarnation eviction of Section II — the forced weight sequence that replaces the crashed identity with a new one — is not exercised by this service. What the demonstration implements is the classic recovery boundary whose amnesia exposure motivates reincarnation, hardened with durable-nonce fencing: the fresh nonce is exactly the freshness ingress that the reincarnation design’s stale-episode rule realises, and a restarting node can never complete recovery with a stale identity stamp. Converting the eviction sequence into this service is the next integration step.

C. The lock and lease API

The client protocol has three operations, one JSON object per line over TCP. Every request carries a message identifier, a stable client identifier, an increasing request number, and a lock identifier; retries reuse the exact envelope, and replication deduplicates by (client, request number), replaying the prior reply. `get` returns the live lease on a lock, or null. `set` grants or renews a lease — a lease identifier, a holder identifier, and an expiry — and a lease is live only when its expiry is later than the execution time; an expired candidate is rejected, an absent or expired incumbent is free, and a live incumbent may be renewed or replaced only by the same holder. `release` removes a live lease only when both holder and lease identifier match, and is idempotent for a missing or expired lease. Lease expiry is what makes the diskless design safe to operate: a crashed holder’s locks free themselves when the lease lapses, without any recovery path trusting the holder’s lost state.

D. Test and hardening status

The service is exercised from several directions: a pure-Teal test suite and an FFI-boundary test suite for the lock service and cluster configuration; a Cerulean format gate and Cyan type checks; the Rust adapter’s checks, tests, and release build, which depend on the exact pinned `vrr-core` git revision fetched over HTTPS; a three-process smoke test covering acquire, read, contention, release, reacquisition, expiry takeover,

and a one-replica restart with a live quorum; and a thirty-second, three-datacentre lease-failover simulation that kills the observed lease holder every three seconds and verifies takeover within five seconds, exiting nonzero on a conflicting holder. The same simulation runs against a stable Docker cluster. CI runs on Linux for regular changes and a full platform matrix with packaged releases for tagged versions.

It is a working implementation under active hardening and testing, not a finished demonstration of the reincarnation protocol.

VIII. EXPERIMENTAL RESULTS: DESIGNS, NOT MEASUREMENTS

WARNING: THIS WORK HAS NOT BEEN DONE. The experiments below are designs; all numbers and charts are placeholders (marked “dummy”).

The claim under test is diskless low latency: rejoin through reincarnation is faster than paying a forced disk flush at the recovery boundary. The decision rule for the campaign: if the measured rejoin latency of E1 is slower than the forced-flush baseline of E2 at the same sample count, the diskless claim collapses — if recovery is slower than doing disk flushes, nothing has been achieved. The system under test for all four designs is the `lunet-locks` demonstration of Section VII; every number below is the placeholder `xxxx`, to be filled at run time.

A. E1: node-kill rejoin latency

The design kills one voting node that is not the leader, rotated across all non-leader voting nodes of the three-datacentre `lunet-locks` cluster, for $k = \text{xxxx}$ kill/rejoin cycles, with continuous competing lock traffic at the lease cadence throughout. Each iteration begins only after the previous rejoin has completed serving and a soak interval has elapsed, so no measurement is contaminated by the previous one. For each iteration the harness waits for steady state, kills the node’s process — Crash-Stop: volatile state lost, the node is by construction a different node on reopen — and the iteration ends when the reincarnated node has rejoined as a weight-0 standby, been walked back to voting weight by the leader’s forced sequence, and is again a voting, serving replica. The metric is the kill-to-rejoin-serving latency, one sample per iteration: p50 `xxxx` ms, p90 `xxxx` ms, p99 `xxxx` ms, max `xxxx` ms, mean `xxxx` ms, and the count of eras and view changes consumed per rejoin, `xxxx`. Fig. 3 shows the designed chart: kill iteration on the horizontal axis, rejoin-serving latency on the vertical axis, one series per milestone of the forced sequence (standby rejoin at weight 0, voting again at weight 1). No data points exist yet; the chart is watermarked “dummy”.

B. E2: disk modes at the recovery boundary

What does the diskless default save, in latency, against paying a forced flush at the recovery boundary? The design builds custom `lunet-locks` binaries, identical in every respect other than the durability behaviour at the recovery

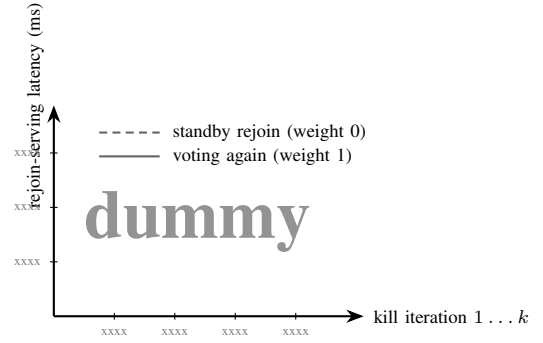


Fig. 3. E1: node-kill rejoin latency.

boundary — the point at which a reincarnating node durably records state before rejoining serving:

- *Diskless (the default)*. No durable writes at the recovery boundary; quorum memory is the only protocol state, and the recovery nonce file remains as shipped.
- *Naive single write (the latency baseline)*. One 4 KiB block write followed by `fsync` at the recovery boundary. Per the FAST’18 finding [3], even a single write plus `fsync` is not fault-safe against real disk faults; the variant is the latency baseline only, never a safety claim.
- *Double-ring write*. A TigerBeetle-style double write at the recovery boundary: one 4 KiB block of data plus a checksum header to the first WAL ring, then the header and checksum with no payload to the second WAL ring, the two rings spaced apart in separate 4 KiB-sector-aligned zones so the two checksum copies sit in different erasure blocks. Both flush variants carry fake data only, with no read-back: they measure write-and-flush latency and never reconstruct state from what was written.

Each variant runs E1’s procedure exactly, $k = \text{xxxx}$ iterations per variant, cold counts. The metrics are the kill-to-rejoin-serving distribution per variant (percentiles as in E1) and, for the flush variants, the recovery-boundary flush latency itself per iteration. The planned report is the rejoin-latency delta: diskless median `xxxx` ms, naive-flush median `xxxx` ms, double-ring median `xxxx` ms, with tail figures `xxxx` ms at the ninth decile. The delta is the utility test: if reincarnation is slower than the forced flush, the diskless design has achieved nothing. Fig. 4 shows the designed chart: node restart event on the horizontal axis, rejoin-serving latency on the vertical axis, one series per build (diskless default, naive single write, double-ring write). No data points exist yet; the chart is watermarked “dummy”.

C. E3: Maelstrom brute force at three and five nodes

Does reincarnation hold consistency under brute-force fault injection? Jepsen’s Maelstrom drives the cluster under its fault profile with reincarnation as the failure model: node crashes with volatile-state loss — the crashed node reincarnates rather than crash-recovers — plus partitions, with a Maelstrom adapter fronting the lock service as a compare-and-set register and competing lock clients at the lease cadence. The design runs the fault profile at three nodes, then at five, `xxxx` runs

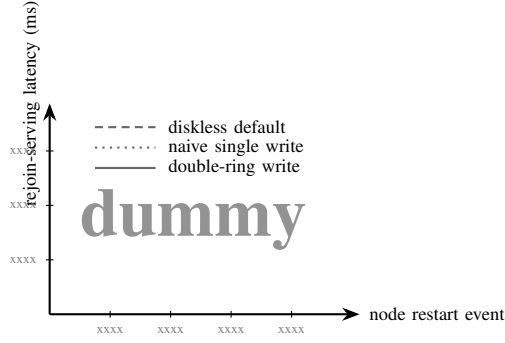


Fig. 4. E2: disk modes at the recovery boundary.

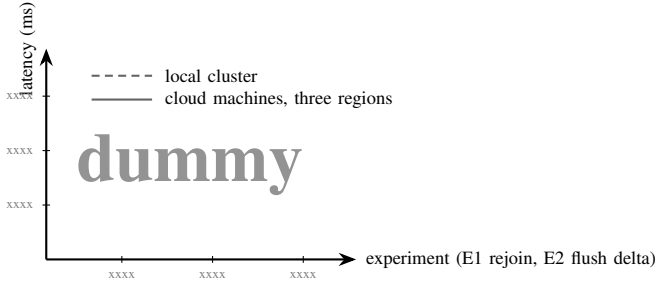


Fig. 5. E4: cloud repetition of E1 and E2.

per cluster size. Three checks decide each run: Maelstrom’s linearizability check passes over the register — no committed operation is lost, and no node rejoins under its old identity to overwrite a newer decision; every killed node reincarnates and rejoins serving, with no run ending in a permanently lost node; and no read ever reports two holders, expiry being judged only in the leader’s clock. All figures xxxx. A passing campaign supports the case that reincarnation eliminates the FAST’18 weakness: consistency under crash with successful rejoin, by construction rather than by recovery.

D. E4: cloud repetition

E1 and E2 are repeated on commodity cloud virtual machines, the six-node cluster laid out over three regions, two machines each — the correlated-infrastructure setting that weighted configurations address. Variables, procedure, metrics, and sample counts are E1’s and E2’s unchanged; only the environment differs. The planned report records xxxx kills per configuration and zone-crossing medians of xxxx ms (E1) and xxxx ms (E2), with xxxx availability incidents. Fig. 5 shows the designed chart: experiment on the horizontal axis (E1 rejoin, E2 flush delta), latency on the vertical axis, one series per deployment (local cluster, cloud machines). No data points exist yet; the chart is watermarked “dummy”.

WARNING: THIS WORK HAS NOT BEEN DONE. The experiments below are designs; all numbers and charts are placeholders (marked “dummy”).

IX. RELATED WORK

Viewstamped Replication introduced the organisation of replication around views and a primary [1]; VRR revisits that

organisation and separates normal operation, view change, recovery and reconfiguration [2]. Michael *et al.* identify the failure of diskless recovery to preserve view-change commitments and construct virtual stable storage [3]. uVRR takes a different route: it makes a crash final for the old identity, then brings the process back as a new member through reconfiguration. The replacement schedule connects that lifecycle to agreement across configurations.

Turner’s *Unbounded Pipelining in Dynamically Reconfigurable Paxos Clusters* supplies the central quorum geometry, era-based agreement argument and casting-vote construction [4]. Withholding the leader’s own promise is Turner’s mechanism. The present work applies it to reincarnation, proves weighted configuration reachability and an abstract casting-vote completion, and proves agreement for a concrete replacement instantiation from the protocol’s history invariants. The application discussion places this design alongside coordination services and CORFU, using their stated semantics and resource context.

REFERENCES

- [1] B. M. Oki and B. H. Liskov, “Viewstamped replication: A new primary copy method to support highly-available distributed systems,” in *ACM PODC*, 1988, pp. 8–17.
- [2] B. Liskov and J. Cowling, “Viewstamped replication revisited,” MIT CSAIL, Tech. Rep. MIT-CSAIL-TR-2012-021, 2012. [Online]. Available: <https://hdl.handle.net/1721.1/71763>
- [3] E. Michael, D. R. K. Ports, N. Kr. Sharma, and A. Szekeres, “Providing stable storage for the diskless crash-recovery failure model,” University of Washington, Tech. Rep. UW-CSE-16-08-02, Aug. 25, 2016. [Online]. Available: <https://syslab.cs.washington.edu/papers/diskless-tr16.pdf>
- [4] D. Turner, “Unbounded pipelining in dynamically reconfigurable Paxos clusters,” revision 1A9DBA37, Aug. 14, 2017. [Online]. Available: <https://github.com/DaveCTurner/paxos-membership>. The proof artifact includes the consulted PDF and its digest.
- [5] S. Massey, “Viewstamped replication revisited,” Aug. 12, 2026. [Online]. Available: <https://simbo1905.wordpress.com/2026/08/12/viewstamped-replication-revisited/>
- [6] M. Balakrishnan, D. Malkhi, J. D. Davis, V. Prabhakaran, M. Wei, and T. Wobber, “CORFU: A Distributed Shared Log,” *ACM Trans. Comput. Syst.*, vol. 31, no. 4, Art. 10, Dec. 2013, doi: 10.1145/2535930.
- [7] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-free coordination for Internet-scale systems,” in *USENIX ATC*, 2010. [Online]. Available: https://www.usenix.org/events/usenix10/tech/full_papers/Hunt.pdf
- [8] etcd authors, “etcd API guarantees,” documentation v3.6, accessed Sep. 11, 2026. [Online]. Available: https://etcd.io/docs/v3.6/learning/api_guarantees/
- [9] etcd authors, “Hardware recommendations,” documentation v3.6, accessed Sep. 11, 2026. [Online]. Available: <https://etcd.io/docs/v3.6/op-guide/hardware/>
- [10] etcd authors, “Embedding etcd in a Go application,” documentation v3.6, accessed Sep. 11, 2026. [Online]. Available: https://etcd.io/docs/v3.6/dev-guide/golang_embed_pkg/
- [11] simbo1905, “The network is faster than the disk,” 12 April 2024, accessed 7 September 2026. <https://simbo1905.wordpress.com/2024/04/12/the-network-is-faster-than-the-disk/>
- [12] simbo1905, “Paxos Voting Weights,” 16 March 2017, accessed 7 September 2026. <https://simbo1905.wordpress.com/2017/03/16/paxos-voting-weights/>
- [13] L. Lamport, D. Malkhi, and L. Zhou, “Reconfiguring a state machine,” *ACM SIGACT News*, vol. 41, no. 1, pp. 63–73, 2010, doi: 10.1145/1753171.1753191.
- [14] L. Lamport, D. Malkhi, and L. Zhou, “Vertical Paxos and primary-backup replication,” in *Proc. 28th ACM Symp. Principles of Distributed Computing (PODC)*, 2009, pp. 312–313, doi: 10.1145/1582716.1582783.
- [15] S. Massey, “uVRR code and proof addendum,” 2026. [Online]. Available: <https://github.com/luca-lunet/uvrr-core>

- [16] L. de Moura and S. Ullrich, “The Lean 4 theorem prover and programming language,” in *Automated Deduction – CADE 28*, 2021, pp. 625–635, doi: 10.1007/978-3-030-79876-5_37.